



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DataMesh Connect: Distributed Open Data Collaboration Platform

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0509 – Database Management Systems

to the award of the degree of

**BACHELOR OF ENGINEERING
IN
BIOINFORMATICS**

Submitted by

Judy Allen J (192520055)

Under the Supervision of

Dr R Kesavan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**DataMesh Connect: Distributed Open Data Collaboration Platform**” has been carried out by **Judy Allen J (192520055)** and under the supervision of **Guide Name** and is submitted in partial fulfilment of the requirements for the current semester of the **B.E.Bioinformatics** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr C Sunitha Ram
Professor
Computer Science and Engineering
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr R Kesavan
Professor
Computer Science and Engineering
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, **Judy Allen J** of the **B.E.Bioinformatics**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**DataMesh Connect: Distributed Open Data Collaboration Platform**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

Judy Allen J (192520055)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical
Sciences Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution
Judy Allen J / 192520055	User interface, dataset workflow, database integration, testing and documentation	Developed both researcher-side workflow and admin-side workflow	Tested registration, login, dataset submission, user isolation and request-status workflows	Introduction, Design, Implementation, Testing, results and conclusion	100%
Total					100%

ABSTRACT

The rapid growth of academic and research data has increased the need for secure and organized platforms capable of supporting data sharing while maintaining appropriate access control. Conventional data repositories often focus primarily on data storage and discovery, while administrative verification, restricted-data access, duplicate detection, user management, and security monitoring may require separate mechanisms. This can result in unauthorized access, inconsistent dataset quality, duplicate submissions, and limited administrative oversight.

DataMesh Connect: Distributed Open Data Collaboration Platform is developed as a secure web-based repository for submitting, storing, discovering, and managing academic datasets. The system provides separate interfaces for researchers and administrators. Researchers can register and authenticate, explore available datasets, submit datasets with relevant metadata, request access to restricted datasets, and monitor the status of their submissions and requests. Administrators are provided with dedicated functionality to review dataset submissions, approve or reject restricted-access requests, manage users, and perform duplicate and security audits. These functions directly reflect the workflows defined in the project presentation.

The application uses React and TypeScript for frontend development, Firebase Authentication for secure identity management, and Cloud Firestore for storing dataset information, access requests, user information, and audit records. GitHub supports source-code management, while Vercel is used for web deployment. Role-based access principles are applied to separate normal users from administrators.

Functional testing was performed for user registration, login, dataset submission, administrative approval, restricted-access requests, request approval, user-data isolation, and administrator access control. All major workflows reported in the project presentation passed their respective tests.

The developed system demonstrates that database-driven access management can improve the security, transparency, and governance of academic dataset sharing while maintaining an accessible workflow for researchers.

Keywords: Open Data Repository, Firebase, Cloud Firestore, Role-Based Access Control, Dataset Sharing, Data Security

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	7
4	CHAPTER 4: Implementation, Testing & Results, Project Management	13
5	CHAPTER 5: Conclusion, Future Work and Reflection	22
	References	25
	Appendices	26

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	Proposed System Architecture	10
Figure 3.2	System Flow Diagram	10
Figure 3.3	Entity Relationship Diagram	11
Figure 4.1	User Registration/Login Interface	14
Figure 4.2	Researcher Dashboard	15
Figure 4.3	Dataset Submission Interface	15
Figure 4.4	Dataset Exploration Interface	16
Figure 4.5	Restricted Access Request Interface	16
Figure 4.6	Administrator Dashboard	17
Figure 4.7	Dataset Approval Interface	17
Figure 4.8	Access Request Approval Interface	18

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing and Proposed Systems	5
Table 3.1	Stakeholder Requirements	7
Table 3.2	Functional and Non-Functional Requirements	7
Table 3.3	Design Specifications	8
Table 3.4	Alternative Solution Evaluation	9
Table 3.5	Trade-off Analysis	11
Table 3.6	Risk Register	12
Table 4.1	Software Tools Used	13
Table 4.2	Test Cases and Results	18
Table 4.3	Achievement of Objectives	20

LIST OF ABBREVIATIONS

Abbreviation	Description
DBMS	Database Management System
UI	User Interface
UID	Unique Identifier
RBAC	Role-Based Access Control
API	Application Programming Interface
URL	Uniform Resource Locator
CRUD	Create, Read, Update and Delete
NoSQL	Not Only SQL
SLO	Service Level Objective
CI	Continuous Integration

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern academic and research activities generate large quantities of digital data that need to be stored, organized, shared, and reused efficiently. Open-data repositories provide an important mechanism through which researchers can make datasets available for further analysis and collaboration. However, not all academic data can be made completely public. Some datasets may require controlled access, administrative validation, or verification before they are released.

DataMesh Connect is designed as a secure platform for storing, sharing, and managing academic datasets. It provides dedicated interfaces for researchers and administrators and supports dataset submission, controlled discovery, restricted-data requests, dataset verification, and administrative monitoring. The system uses Firebase Authentication and cloud-based data services for user identity and database operations.

The platform therefore combines the concepts of an open-data repository with access management and administrative governance.

1.2 Need for the Project

The increasing quantity of academic datasets creates several data-management challenges. Researchers require a centralized location through which datasets can be uploaded, discovered, and shared. At the same time, institutions must ensure that restricted information is not exposed to unauthorized users.

Major problems addressed by this project include:

- Unauthorized access to restricted datasets.
- Lack of clear separation between normal users and administrators.
- Dataset submissions without administrative validation.
- Duplicate or potentially unsafe submissions.
- Difficulty tracking access requests and their approval status.
- Lack of centralized monitoring and audit information.
- Difficulty maintaining accountability for dataset sharing.

Therefore, a controlled repository with role-based permissions, dataset approval, and request-based access is required.

1.3 Problem Statement

Existing academic data-sharing systems may provide basic storage and dataset retrieval but often lack integrated mechanisms for secure access control, dataset verification, restricted-access management, duplicate detection, and administrative monitoring.

The engineering problem is therefore to design and develop a secure database-driven repository capable of allowing researchers to submit and discover datasets while simultaneously enforcing role-based permissions, administrative approval, user-data isolation, and controlled access to restricted datasets.

The system must balance two potentially conflicting requirements: easy data accessibility for researchers and strong governance and security for administrators.

1.4 Project Aim

The aim of this project is to design and develop a secure open-data collaboration platform that enables researchers to submit, discover, and request datasets while providing administrators with effective mechanisms for access control, dataset approval, security monitoring, and repository governance.

1.5 Project Objectives

1. To design and develop a secure repository for submitting, storing, discovering, and sharing academic datasets with clear status tracking.
2. To implement role-based access through separate researcher and administrator interfaces with permission-controlled actions.
3. To implement dataset approval/rejection, restricted-access requests, duplicate detection, and security auditing for improved data governance.

1.6 Scope

The scope of DataMesh Connect includes:

Included

- User registration and authentication.
- Researcher login.
- Administrator authentication.
- Separate user and administrator dashboards.
- Dataset submission with metadata.
- Public and restricted dataset classification.
- Dataset search and exploration.
- Restricted dataset access requests.

- Administrative approval/rejection.
- Dataset submission-status tracking.
- Access-request status tracking.
- User management.
- Duplicate checking.
- Security/audit monitoring.
- Cloud-based data storage.
- Role-based access enforcement.

Excluded

- Automatic AI-based dataset classification.
- Large-scale scientific data processing.
- Complete visualization and statistical analysis of uploaded datasets.
- Native mobile application.
- Integration with external institutional repositories.

These excluded features may be considered as future extensions.

1.7 Expected Engineering Outcome

The expected outcome is a functional web-based prototype of a secure academic dataset repository.

The completed system is expected to:

- Authenticate users securely.
- Maintain separate researcher and administrator functionality.
- Store dataset records efficiently.
- Allow users to submit and discover datasets.
- Control access to restricted datasets.
- Allow administrators to approve or reject submissions and requests.
- Maintain appropriate request and audit records.
- Protect administrator functionality from unauthorized users.
- Maintain transparency through dataset and request statuses.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted by examining existing concepts related to open-data repositories, database-management systems, data-access control, data-sharing standards, and web-based database applications. Academic literature, database textbooks, technical documentation, and data-sharing best-practice recommendations were considered.

The primary sources referenced during the project include DBpedia research, Database System Concepts, Fundamentals of Database Systems, database documentation, and W3C Data on the Web Best Practices.

2.2 Review of Existing Work

Open Data Repositories

Open-data repositories provide centralized locations where datasets can be published and discovered. Their major advantage is improved accessibility and reuse of information. However, research datasets may contain information that should not be freely accessible to every user, creating the need for more controlled sharing mechanisms.

DBpedia

DBpedia demonstrates the usefulness of structured and interconnected open data. It provides a model for publishing and accessing structured information on a large scale. However, the primary purpose of DataMesh Connect differs because the proposed system concentrates on academic dataset submission, administrative approval, and restricted-access workflows.

Database Management Systems

Database-management principles described by Silberschatz, Korth and Sudarshan and by Elmasri and Navathe provide the foundation for structured data storage, relationships, access management, consistency, and database operations.

Data on the Web Best Practices

W3C data-sharing practices emphasize discoverability, metadata, accessibility, and appropriate data publication. These concepts are relevant to DataMesh Connect because dataset records need meaningful metadata and controlled accessibility.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
General Open Data Repository	Easy dataset discovery and sharing	Limited control for sensitive/restricted datasets	Provides the basic repository concept
DBpedia/Open Linked Data	Large-scale structured data accessibility	Not designed around individual restricted-access approval	Demonstrates structured data sharing
Conventional Local Database System	Strong structured storage and management	Limited distributed accessibility	Provides DBMS principles
DataMesh Connect	Secure authentication, role separation, approval workflow, audits and controlled access	Prototype currently lacks advanced AI/large-file features	Integrates repository and access management

2.4 Research / Engineering Gap

Existing data-sharing approaches may emphasize either accessibility or database management. A practical engineering gap exists when institutions require both easy dataset discovery and strong administrative governance within the same system.

Important unresolved requirements include:

- Controlled access to restricted datasets.
- Transparent request/approval tracking.
- Separation between researchers and administrators.
- Verification before dataset publication.
- Duplicate-dataset detection.
- Security monitoring.
- User-specific record isolation.

DataMesh Connect addresses these requirements through an integrated database-driven workflow.

2.5 Proposed Contribution

The proposed system combines open-data discovery with controlled administrative management.

The major contributions are:

- Separate researcher and administrator portals.
- Dataset submission and administrative verification.
- Public and restricted access types.
- Restricted dataset request workflow.
- Access-request approval and rejection.
- Dataset and request status tracking.
- User-data isolation.
- Duplicate checking.
- Security and activity auditing.
- Cloud-based database implementation.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Researcher/User	Register and securely log in
Researcher/User	Explore available datasets
Researcher/User	Submit datasets and metadata
Researcher/User	Request access to restricted datasets
Researcher/User	Track submission and request status
Administrator	Secure administrator login
Administrator	Review dataset submissions
Administrator	Approve or reject datasets
Administrator	Approve or reject restricted-access requests
Administrator	Manage users
Administrator	Perform security and duplicate audits

Functional and Non-Functional Requirements

Requirement	Type	Measurable Target
User registration	Functional	Valid user account successfully created
User authentication	Functional	Valid credentials provide access
Dataset submission	Functional	Dataset saved with Pending status
Dataset exploration	Functional	Approved/public datasets displayed
Access request	Functional	Request stored and visible to admin
Admin approval	Functional	Admin can approve/reject request
User isolation	Functional	Users access only permitted records
Role separation	Functional	Normal users cannot access admin functions

Security	Non-Functional	Access controlled using authentication and rules
Reliability	Non-Functional	Core workflows execute without failure
Scalability	Non-Functional	Cloud database supports increasing records
Usability	Non-Functional	Clear researcher/admin interfaces

3.2 Constraints & Applicable Standards

Standard / Guideline	Requirement	How Applied in This Project
W3C Data on the Web Best Practices	Improve accessibility, metadata quality and discoverability	Dataset metadata and structured repository records are maintained
Authentication and authorization principles	Restrict system actions according to authenticated identity	Firestore Authentication and role separation are used
Database integrity principles	Maintain consistent relationships between records	Dataset, user and request identifiers link related records
Data minimization principles	Store only information required for repository functionality	User and dataset records contain application-relevant fields

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
User Authentication	Required	Boolean	High
Admin/User Separation	Complete separation	Role	High
Dataset Access Types	Public/Restricted	Category	High
Dataset Status	Pending/Approved/Rejected	Status	High
Access Request Tracking	Enabled	Status	High
Duplicate Detection	Enabled	Boolean	Medium
Audit Monitoring	Enabled	Record	Medium
Cloud Database	Firestore	Platform	High
Deployment	Web-based	Application	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Traditional Local SQL Application

A locally hosted relational database could provide structured data storage and SQL functionality. However, additional backend infrastructure would be required to provide cloud authentication, remote access, and deployment.

Alternative 2: Cloud Application with Firebase

Firebase provides authentication and cloud database functionality that can be integrated directly with the web application. It supports fast implementation of authentication and cloud-connected records.

Alternative 3: Basic Repository Without Role Separation

A simple repository could allow users to upload and browse datasets without separate administrator control. Although easier to develop, it would not meet the security and governance requirements of this project.

Criterion	Weight	Traditional SQL	Firebase-Based System	Basic Repository
Performance	25%	4	5	4
Cost	20%	3	4	5
Security	25%	4	5	2
Scalability	15%	3	5	3
Reliability	15%	4	5	3

Selected Alternative: Firebase-Based Web Application

3.5 Selected Design & Detailed Design

The Firebase-based architecture was selected because it directly supports the major functional requirements of the project, including authenticated identities, cloud-connected data storage, scalable repository records, and user-specific access.

The overall system consists of three major components:

Researcher Portal → Firebase Services → Administrator Portal

The researcher portal allows registration, authentication, dataset exploration, submission, and restricted-access requests. The administrator portal receives relevant submissions and requests and performs approval, rejection, user management, security audits, and duplicate checks. The backend uses Firebase Authentication and Cloud Firestore to support the workflows.

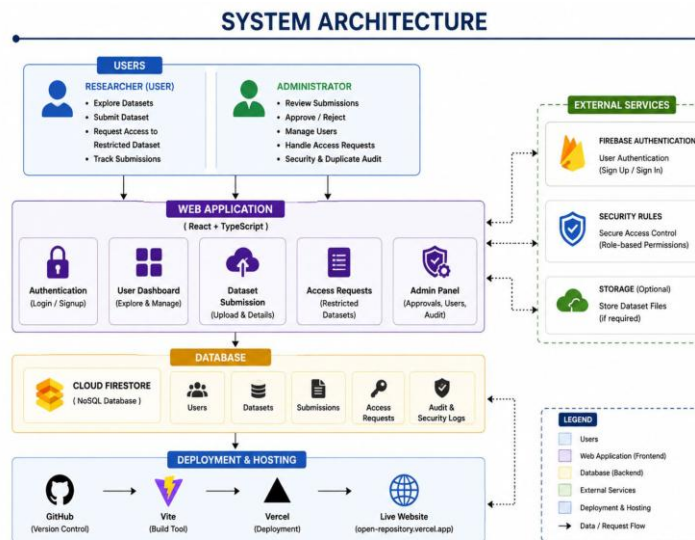


Figure 3.1: System Architecture of DataMesh Connect

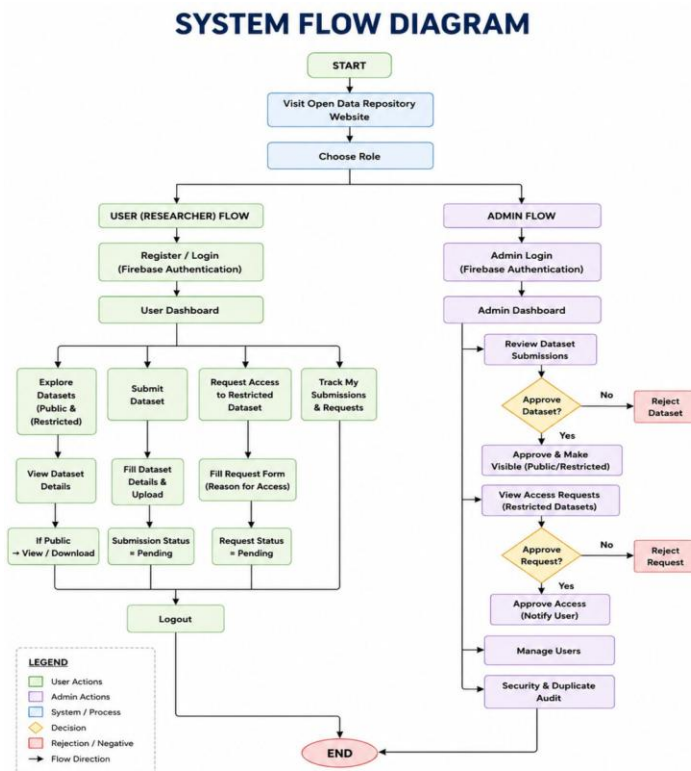


Figure 3.2: DataMesh Connect System Workflow

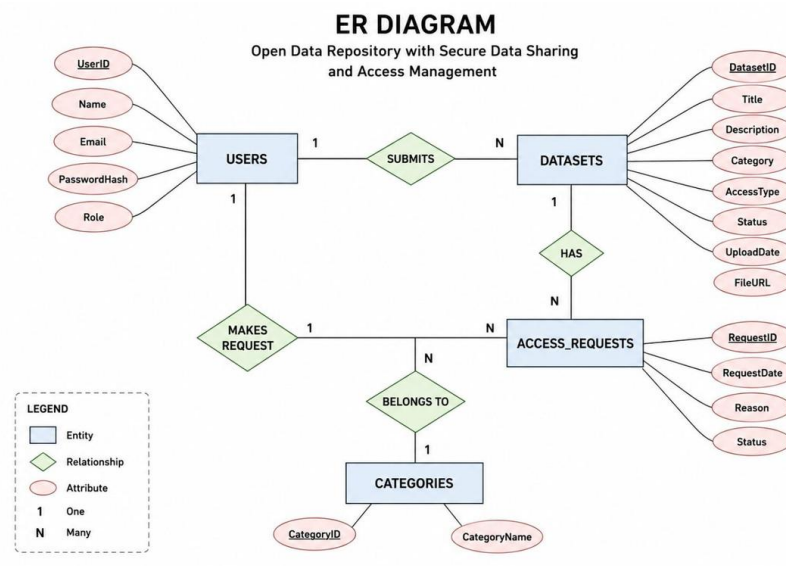


Figure 3.3: Entity Relationship Diagram

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Firebase Authentication	Custom authentication	Secure and easier integration	Dependency on cloud service	Firebase selected for reliable identity management
Cloud Firestore	Local relational DB	Cloud availability and scalability	NoSQL structure differs from traditional SQL	Firestore selected for web application integration
Separate admin portal	Shared dashboard	Improved role security	Additional UI development	Separate portal selected
Approval workflow	Direct publication	Better data governance	Additional review step	Approval workflow selected
Restricted datasets	Public-only repository	Controlled information sharing	Requires request management	Restricted access supported

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Cloud-based infrastructure reduces need for dedicated local hardware	Firebase and serverless deployment architecture	Cloud-based deployment selected
Ethics	Dataset access should respect ownership and permissions	Restricted/public access classification	Approval workflow implemented
Health & Safety	Project is software-based and creates no significant direct physical hazards	Web-based development environment	No specialized physical safety controls required
Security	Unauthorized access and privilege escalation must be prevented	Role separation and authentication requirements	Separate admin/user authentication adopted

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Unauthorized admin access	Medium	High	High	Role-based authentication and security rules	Low
Duplicate dataset submission	Medium	Medium	Medium	Duplicate audit	Low
Malicious dataset content	Low	High	Medium	Security review/auditing	Low-Medium
Accidental data exposure	Low	High	Medium	Public/restricted classification	Low
Incorrect request approval	Low	Medium	Medium	Admin review and status tracking	Low
Database service interruption	Low	Medium	Low-Medium	Cloud service architecture	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative web-development approach. The basic user and administrator interfaces were developed first, followed by authentication, dataset storage, access-request handling, administrative approval, and security-related functionality. Each feature was individually tested before being integrated into the complete workflow.

The application uses a cloud-connected architecture where researcher and administrator actions communicate with Firebase services. Database records are stored in Cloud Firestore and linked through unique identifiers.

4.2 Software Implementation & Modern Tools Used

The technologies listed in your PPT are React + TypeScript, Vite, Firebase Authentication, Cloud Firestore, GitHub, and Vercel.

Tool / Software	Purpose	Stage Used
React	Frontend user-interface development	Development
TypeScript	Type-safe application logic	Development
Vite	Development and build environment	Development
Firebase Authentication	User registration/login and identity	Authentication
Cloud Firestore	Dataset, access request and audit storage	Database
GitHub	Source-code version control	Development
Vercel	Web application deployment	Deployment

4.2.1 Firebase Database Connection

```
const app = initializeApp(firebaseConfig);
```

```
const auth = getAuth(app);
```

```
const db = getFirestore(app);
```

Purpose: Establishes the application connection with Firebase Authentication and Cloud Firestore.

4.2.2 Dataset Submission

```
await addDoc(collection(db, "datasets"), {  
  title: datasetTitle,  
  ownerId: currentUser.uid,  
  accessType: accessType,  
  status: "Pending"  
});
```

Purpose: Stores a newly submitted dataset and assigns the initial status as Pending until administrator review.

4.2.3 Restricted Access Request

```
await addDoc(collection(db, "accessRequests"), {  
  datasetId: datasetId,  
  userId: currentUser.uid,  
  status: "Pending"  
});
```

Purpose: Stores a request made by a user to access a restricted dataset.

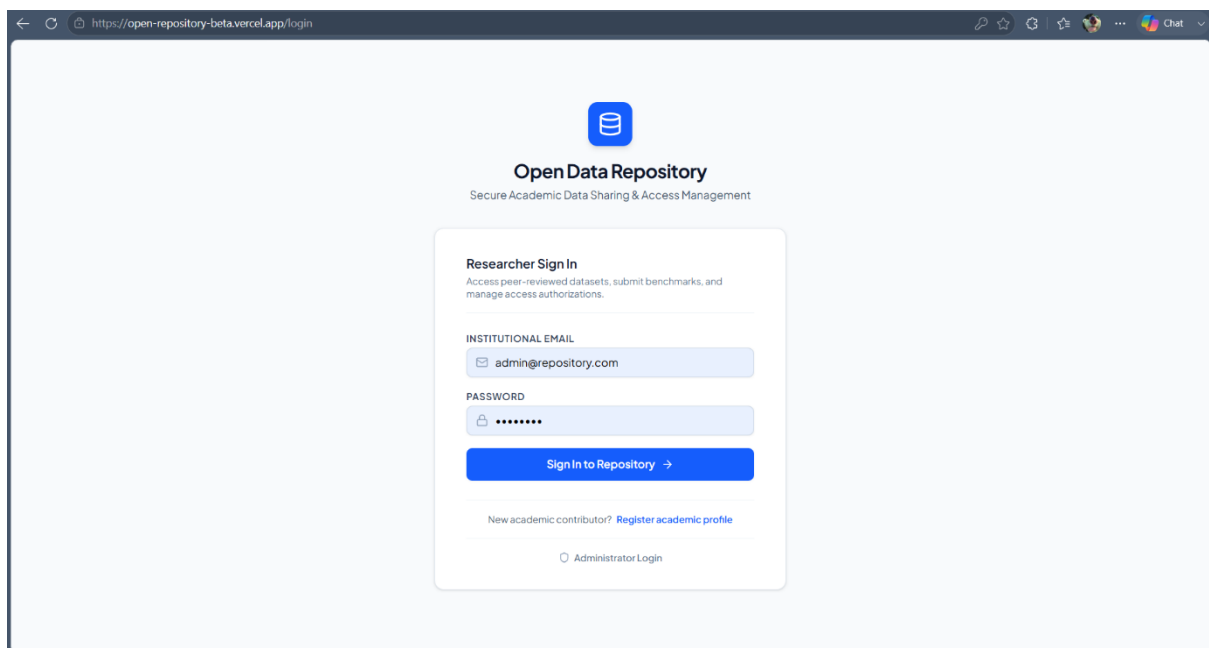


Figure 4.1 – User Registration/Login Screen

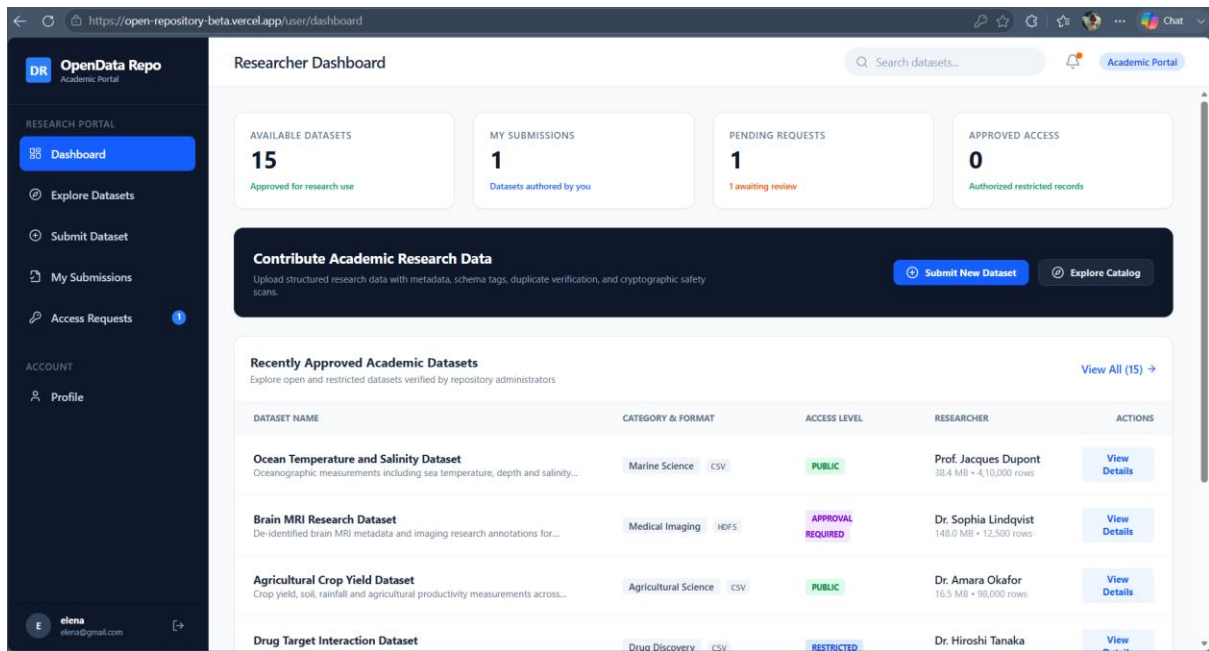


Figure 4.2 – Researcher Dashboard

The screenshot displays the 'Dataset Submission Portal' of the 'OpenData Repo Academic Portal'. The left sidebar is identical to the previous screenshot. The main content area features a 'Submit Academic Dataset' section with a description: 'Submit your experimental, benchmark, or observational dataset for review and publication. All submissions undergo administrative duplicate verification and security audit prior to publication.' Below this is a 'DATASET IDENTIFICATION' section with the following fields:

- DATASET TITLE ***: e.g., Deep Learning Benchmark for High-Resolution Satellite Imagery
- DESCRIPTION / ABSTRACT ***: Provide a comprehensive academic abstract explaining the dataset context, methodology, experimental parameters, and scientific significance...
- ACADEMIC CATEGORY ***: Computer Science & AI
- ACCESS CLASSIFICATION ***: Public (Open Academic Access)

Below the identification section is a 'TECHNICAL FILE SPECIFICATIONS' section with the following fields:

- FILE FORMAT ***: CSV
- FILE NAME / INFO ***: e.g. satellite_imagery_benchmark_v2.csv
- FILE SIZE ***: e.g. 48.2 MB
- RECORD COUNT / OBSERVATIONS ***: e.g. 150000 or 150,000
- KEYWORDS (COMMA-SEPARATED) ***: e.g. deep learning, satellite, remote sensing, segmentation

Figure 4.3 – Dataset Submission

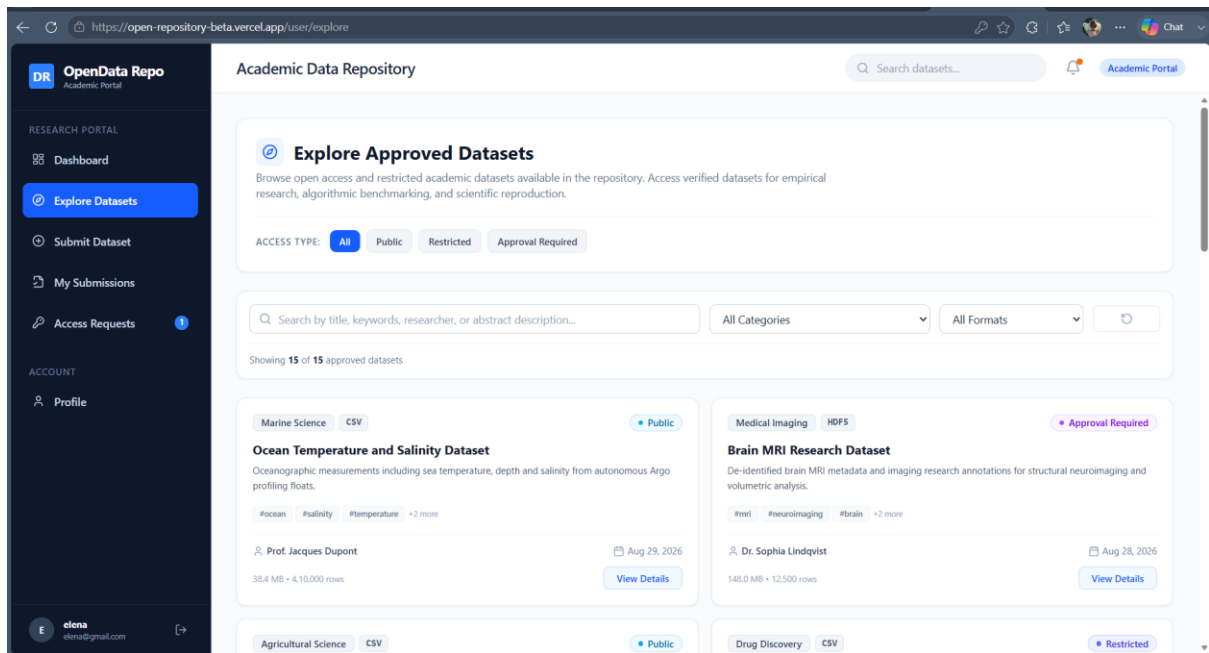


Figure 4.4 – Explore Datasets

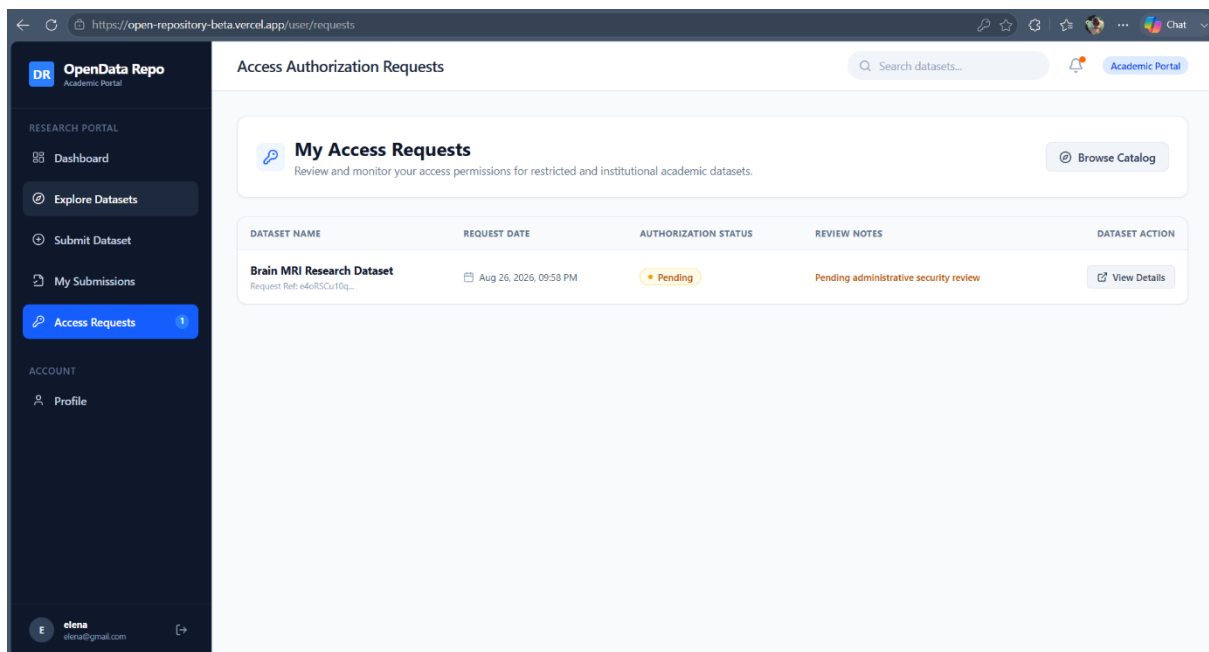


Figure 4.5 – Restricted Access Request

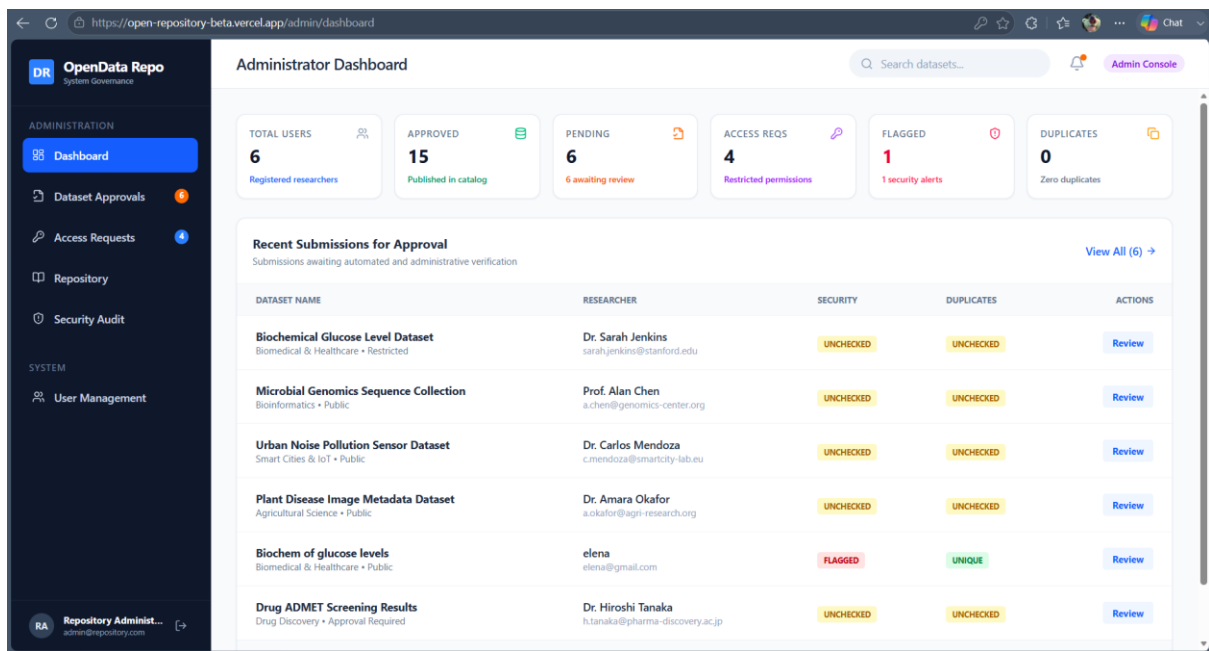


Figure 4.6 – Administrator Dashboard

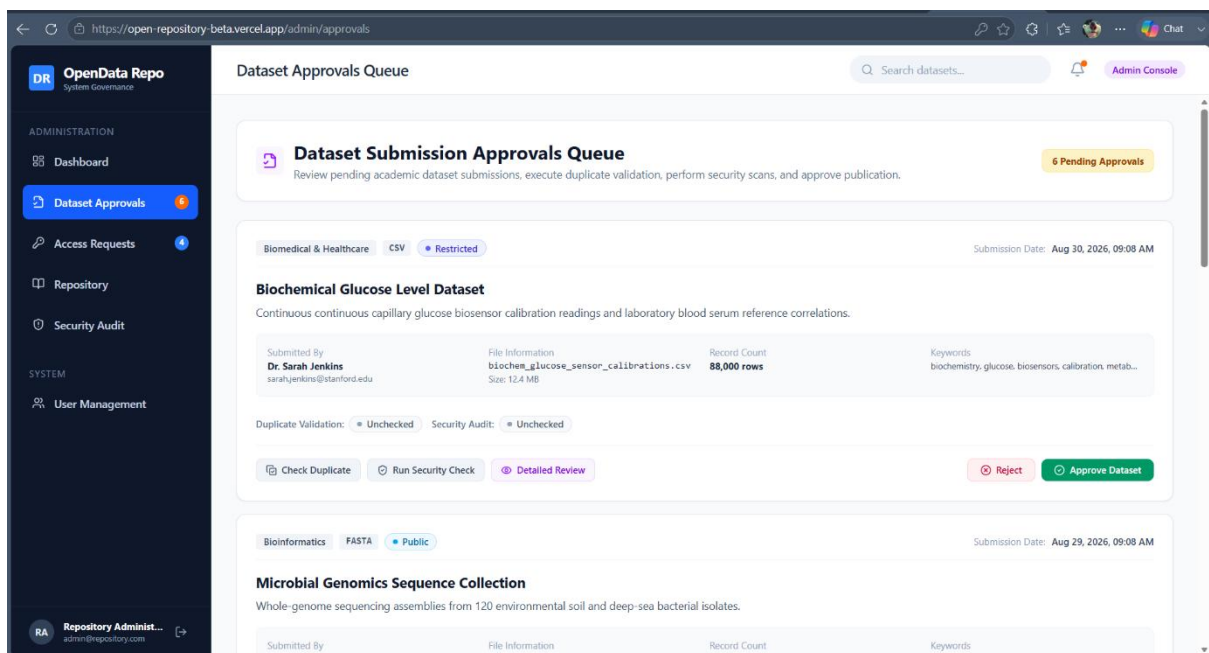


Figure 4.7 – Dataset Approval

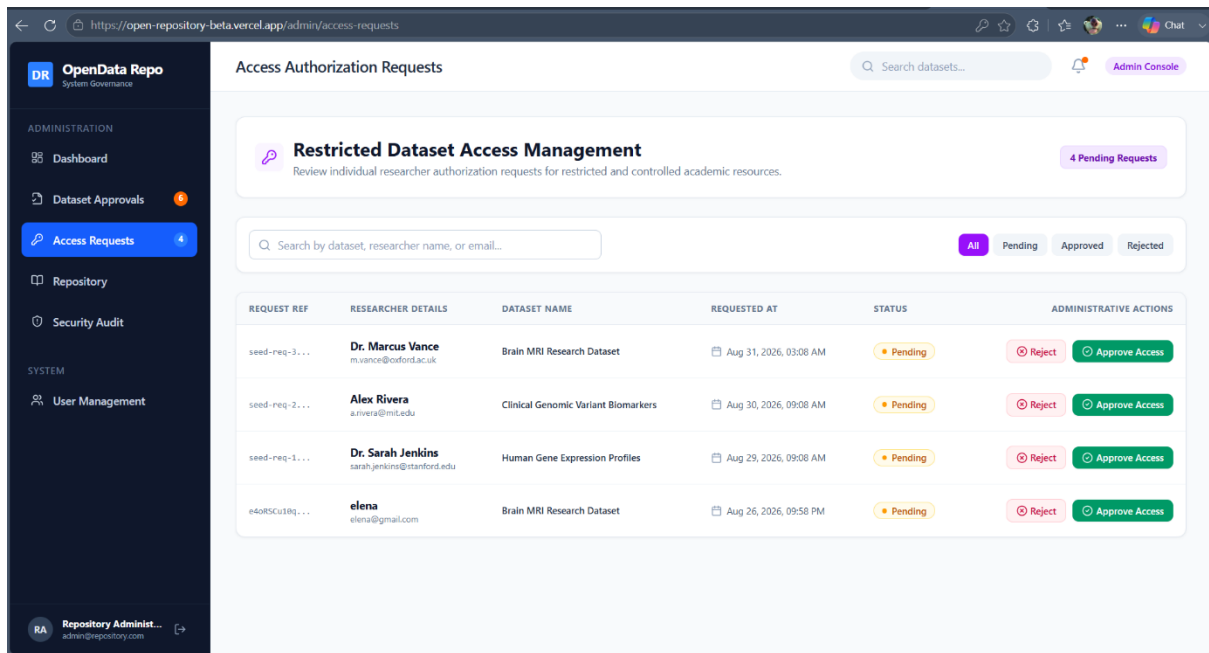


Figure 4.8 – Access Request Approval

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
User registration	Register using valid user details	Account successfully created
Login	Enter valid credentials	Dashboard displayed
Dataset submission	Submit valid dataset	Record stored with Pending status
Admin approval	Approve/reject submitted dataset	Status successfully updated
Restricted access	Submit request	Request visible to administrator
Request approval	Admin approves request	User-side status updated
Data isolation	Login through different accounts	Users see only authorized data
Admin protection	Access admin functionality as normal user	Access denied

Verification Results

Specification	Required Value	Achieved Value	Status
Registration	Successful	Successful	PASS
Login	Successful	Successful	PASS
Dataset Submission	Pending record created	Pending record created	PASS
Dataset Approval	Admin approval functional	Functional	PASS
Restricted Request	Visible in Admin Portal	Visible	PASS

Access Approval	User status updated	Updated	PASS
User Isolation	Restricted to own records	Achieved	PASS
Admin Access Control	Users blocked from admin functions	Achieved	PASS

4.4 Results

The implementation produced a functional prototype of DataMesh Connect capable of supporting the major researcher and administrator workflows.

The system successfully demonstrated:

- User registration and authentication.
- Researcher and administrator role separation.
- Dataset submission.
- Pending dataset status.
- Administrator dataset review.
- Public and restricted dataset management.
- Restricted-access request generation.
- Request approval and status synchronization.
- User-specific record isolation.
- Administrative access protection.
- Duplicate/security audit functionality.

The results indicate that the proposed architecture can support controlled academic dataset sharing while maintaining administrative governance.

4.5 Data Analysis & Engineering Judgment

Testing demonstrated successful operation of all major functional workflows defined for the prototype. In particular, role separation and request-status synchronization were important indicators of correct system behavior.

The use of unique user identifiers enables dataset and request records to be associated with specific researchers. Similarly, linking access requests with dataset identifiers allows one restricted dataset to receive multiple requests while maintaining traceability.

A significant engineering consideration was the balance between usability and security. Direct access to every dataset would simplify the researcher experience but would eliminate

control over sensitive datasets. Therefore, the public/restricted classification and administrator approval workflow were maintained.

Cloud Firestore also provides a flexible database structure suitable for rapidly developing the prototype. However, database queries, indexing, and read/write operations should be optimized if the number of users and datasets increases substantially.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop a secure academic dataset repository	User authentication, dataset submission and repository interface implemented	Fully Achieved
Implement researcher/admin role separation	Dedicated researcher and administrator interfaces implemented	Fully Achieved
Implement approvals, access requests and audits	Dataset approval, restricted requests, duplicate/security checks implemented	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Secure Firebase-based authentication.
- Separate researcher and administrator interfaces.
- Controlled restricted-data access.
- Transparent approval workflow.
- User-specific record isolation.
- Dataset moderation before publication.
- Duplicate-check capability.
- Security/audit records.
- Cloud-based scalable database.
- Simple and accessible web interface.

Limitations

- The current version is primarily a prototype.
- Large dataset files require more specialized cloud file storage.
- Advanced dataset visualization is not currently implemented.

- AI-driven dataset recommendation is not currently available.
- Email approval notifications are not yet implemented.
- External research-platform APIs are not integrated.
- Continuous enterprise-level security monitoring would require additional infrastructure.

4.8 Project Planning, Roles & Budget

Milestone Table

Phase	Activity	Outcome
Phase 1	Problem identification	Project requirements finalized
Phase 2	Database and system design	Schemas and workflows created
Phase 3	User module development	Researcher portal completed
Phase 4	Admin module development	Admin portal completed
Phase 5	Firebase integration	Authentication and Firestore connected
Phase 6	Testing	Major workflows verified
Phase 7	Deployment	Web application deployed
Phase 8	Documentation	Report and presentation completed

Roles and Contributions

Student	Assigned Responsibility	Actual Contribution
Judy Allen J	User module, database workflow and documentation, Admin module, Testing/database	User interface development, submission/access workflows, Firebase integration, testing and report preparation, Admin interface and approvals and Database verification, debugging and documentation

Budget

Item	Estimated Cost	Actual Cost
React / TypeScript	₹0	₹0
Firebase Development Services	₹0	₹0
GitHub	₹0	₹0
Vercel Deployment	₹0	₹0
Development Tools	₹0	₹0
Total	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

DataMesh Connect was developed to address the requirement for a secure and governed academic data-sharing platform. The system integrates dataset storage and discovery with role-based access management, administrative verification, restricted-access requests, duplicate checking, and security auditing.

Separate researcher and administrator interfaces were developed to ensure clear responsibility and reduce unauthorized privilege access. Researchers can register, authenticate, explore datasets, submit datasets, request restricted access, and monitor statuses. Administrators can review submissions and access requests, perform approvals or rejections, manage users, and conduct security-related audits.

The implementation uses React and TypeScript for the frontend, Firebase Authentication for identity management, and Cloud Firestore for data storage. Functional testing confirmed successful operation of the major workflows, including registration, login, submission, approval, restricted access, user isolation, and administrator access protection.

Overall, the project demonstrates that a cloud-based database system with controlled permissions and administrative governance can provide a practical foundation for secure academic dataset collaboration. This matches the conclusion of your PPT, which describes the repository as a secure and auditable platform balancing researcher usability with administrator controls.

5.2 Future Work

Future enhancements can include:

1. AI-driven dataset recommendations for suggesting relevant datasets based on user interests.
2. Advanced visualization modules for exploratory analysis.
3. Email notifications for approvals, rejections, and access requests.
4. Dedicated cloud file storage for large dataset files.
5. External APIs for integration with research platforms.
6. Mobile application development for dataset submission and approval.
7. Enhanced security monitoring and anomaly detection.
8. Dataset version control.
9. Automatic metadata extraction.
10. Advanced search and recommendation features.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Cloud-based database design.
- Firebase Authentication.
- Cloud Firestore integration.
- Role-based access control.
- Research dataset governance.
- Restricted-access workflow development.
- Frontend/database integration.
- Deployment and version-control concepts.

Engineering & Professional Skills Developed

- Database modelling.
- Requirement analysis.
- Web application development.
- Debugging.
- Functional testing.
- Security-aware application design.
- Technical documentation.
- Team coordination.
- Project presentation.

Individual Reflection

During the development of DataMesh Connect, one of the most important challenges was ensuring that researcher and administrator functionality remained clearly separated while still communicating through the same database. I contributed to designing and testing the user-side workflow, including dataset submission, restricted-access requests, and status tracking.

A key engineering decision was to connect user records with unique identifiers so that datasets and requests could be associated with the correct user while preserving data isolation. Testing helped identify workflow and permission issues and made it possible to improve the reliability of the final implementation.

Through the project, I gained practical knowledge of Firebase Authentication, Cloud Firestore, role-based permissions, database relationships, frontend-backend communication, debugging, and deployment. If the project were extended, I would focus on improving automated security monitoring, large-file handling, notification services, and advanced dataset-search capabilities.

REFERENCES

- [1] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, “DBpedia: A nucleus for a web of open data,” in *The Semantic Web*, ISWC 2007, pp. 722–735, 2007.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.
- [3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [4] Oracle Corporation, *MySQL 8.4 Reference Manual*, 2025.
- [5] World Wide Web Consortium, *Data on the Web Best Practices*, W3C.
- [6] Google Firebase, *Firebase Authentication Documentation*.
- [7] Google Firebase, *Cloud Firestore Documentation*.
- [8] Meta Platforms, Inc., *React Documentation*.
- [9] GitHub, Inc., *GitHub Documentation*.
- [10] Vercel Inc., *Vercel Documentation*.

APPENDICES

APPENDIX A

ADDITIONAL DATABASE DETAILS

The detailed system design, architecture, workflow, ER diagram, implementation code, screenshots, testing results, and output evidence have already been presented in **Chapters 3 and 4**. Therefore, this appendix contains only supplementary database information that supports the implementation.

A.1 Database Collections

The DataMesh Connect system uses four major data collections:

Collection	Purpose
Users	Stores registered user information and role details
Datasets	Stores dataset metadata, ownership, access type and approval status
Access Requests	Stores requests submitted for restricted datasets
Audit / Security	Stores security, duplicate-check and activity information

These correspond to the database structures defined in your project presentation.

A.2 Database Relationships

The major relationships used in the system are:

- One user can submit multiple datasets.
- One user can create multiple access requests.
- One restricted dataset can receive multiple access requests.
- One dataset can have multiple audit or security records.

The same cardinalities are specified in the project design.

APPENDIX B

ENGINEERING DRAWINGS / SCHEMATICS

The major engineering diagrams of the project, including the System Architecture Diagram, System Flow Diagram, and Entity Relationship Diagram, have already been presented and discussed in Chapter 3: Engineering Design & Trade-offs.

Reference:

- System Architecture – Section 3.5
- System Flow Diagram – Section 3.5
- ER Diagram – Section 3.5

APPENDIX C**SOURCE CODE / CODE REPOSITORY INFORMATION**

Essential implementation code excerpts, including Firebase initialization, dataset submission, and restricted-access request handling, have already been presented in Chapter 4: Implementation, Testing & Results.

The complete source code of DataMesh Connect is maintained in the project GitHub repository.

GitHub Repository:

[Allen08-Commits/Open_repository: app](#)

The repository contains the complete source code required to build and run the application, including frontend components, authentication logic, Firestore integration, dataset-management functionality, access-request workflows, and administrator operations.

This approach is actually aligned with the report template, which recommends keeping only essential code excerpts in the report and maintaining the complete source separately in the approved repository.

APPENDIX D**DATASHEETS****Not Applicable.**

DataMesh Connect is a software-based database application and does not involve electronic hardware or physical components requiring technical datasheets.

APPENDIX E**EXPERIMENTAL / RAW DATA**

The functional test cases, expected results, achieved results, and execution screenshots have already been presented in Sections 4.3 and 4.4.

No separate experimental raw dataset was generated because the project primarily involved functional verification of a web-based database application.

The major workflows tested included:

- User registration
- User authentication
- Dataset submission
- Dataset approval/rejection
- Restricted-access requests
- Access-request approval
- User-data isolation
- Administrator access control

APPENDIX F

ETHICS / INSTITUTIONAL APPROVAL

Not Applicable.

The present prototype does not involve human experimentation, animal experimentation, clinical trials, or collection of patient data requiring institutional ethical approval.

However, ethical principles related to privacy, controlled dataset access, responsible data sharing, and prevention of unauthorized access were considered during system design.

APPENDIX G

PROJECT PROGRESS RECORD

Stage	Activity	Outcome
1	Problem identification	Secure academic dataset-sharing problem identified
2	Requirement analysis	Researcher and administrator requirements finalized
3	Database design	Main entities and relationships designed
4	System design	User/admin workflows and architecture developed
5	Researcher portal development	User functions implemented
6	Administrator portal development	Administrative functions implemented

7	Firebase integration	Authentication and Firestore integrated
8	Functional integration	Dataset and request workflows connected
9	Testing and debugging	Core workflows verified and corrected
10	Deployment	Application prepared for web deployment
11	Documentation	Presentation and final report completed

Project Progress Summary

The project was developed through successive stages beginning with problem identification and requirement analysis. Database structures and system workflows were designed before implementing the researcher and administrator interfaces. Firebase Authentication and Cloud Firestore were subsequently integrated with the application. Functional testing and debugging were performed to verify the major system operations. The final stages involved deployment, presentation preparation, and technical documentation.

APPENDIX H

USER / INDUSTRY FEEDBACK

Not Applicable.

Formal external user or industry feedback was not collected as part of the current academic prototype.

APPENDIX I

PUBLICATIONS / PATENTS / COMPETITIONS / PRODUCT OUTCOMES

Not Applicable.

No publication, patent, competition submission, or commercial product outcome has been generated from this project at the time of report submission.

APPENDIX J

INDIVIDUAL CONTRIBUTION EVIDENCE

DataMesh Connect was developed as an **individual project** by:

Name: Judy Allen J

Register Number: 192520055

I independently carried out the major stages of the project, including:

- Problem identification and requirement analysis
- Database design
- System architecture and workflow design
- Researcher interface development
- Administrator interface development
- Firebase Authentication integration
- Cloud Firestore integration
- Dataset submission workflow
- Restricted-access request workflow
- Dataset and request approval functionality
- User-data isolation
- Duplicate and security-audit functionality
- Functional testing
- Debugging
- GitHub repository management
- Deployment
- Presentation preparation
- Final report documentation

Individual Contribution Statement

I independently designed, developed, integrated, tested, and documented the DataMesh Connect platform. The project allowed me to apply database-management concepts including data modelling, entity relationships, authentication, access control, data isolation, status management, and database-driven application development.

Through the project, I gained practical experience with React, TypeScript, Firebase Authentication, Cloud Firestore, GitHub, application testing, debugging, and web deployment, which are also the main technologies identified in the project presentation.